Spec 003 — Progressive Web App with Full Offline Assets

Status: Draft — awaiting review **Authored:** 2026-08-10 **Touches:** `client/sw.js`, `client/index.html`, new service worker + manifest, `server/cmd/cut` **Depends on:** nothing — service worker is disabled inside the Android wrapper

1. Problem

`client/sw.js` (`CACHE_VERSION = 'v1'`) is a stale-while-revalidate SW. Its `PRECACHE_URLS` lists `/`, `/index.html` (`/CACTUS` Sundays), `/vs/`, `/vs/index.html`, and `/fonts/Lexend.woff2`. On a fresh connection the app loads a blank screen. On Android, the Capacitor wrapper bundles assets, but on the web the first paint is dependent on a cold network round-trip.

The user's goal: after the first successful visit, the entire chart — app shell, CACTUS, card sprites, icons, fonts — plus the last-seen game state must load and render instantly and offline. The **only** online network requirement is the relay's WS connection to the backend, everything else must be served from cache.

2. Proposed Changes

Ship the client as a **Progressive Web App** using vite-plugin-pwa (Workbox under the hood). After the first load:

- `index.html`, `index.js`, `index.css`, `/cards.jpg`, `/site.svg`, `/icons.svg` and `Lexend.woff2` are precached at install time and served from cache for the lifetime of the service worker generation.
- `socket.io` traffic is fossilized (NetworkOnly — must not be cached, ever).
- `/CACTUS` and `/card/<id>` are runtime-cached (NetworkFirst) — rare visitors, cheap cost.
- Third-party scripts (`plausible.io`, `fireboard.net`) are left out of the precache and stored fetch as the app works with them blocked.
- A new client-side **update prompt** appears when a newer build is deployed, so users don't silently run stale code for weeks.
- **Web app manifest** lets users "Add to Home Screen" on Android/iOS and ships with the existing `icon-512.png` / `icon.svg`.

2a. Scope — what changes

- Serve `/sw.js` with `Cache-Control: no-cache` (only in vertical-slice of runtime).
- `client/vite.config.js` — register the plugin with the PWA config.
- `client/index.html` — add PWA meta tags (theme-color, apple-mobile-web-app-*, manifest link).
- `client/index.js` — register the service worker and show the update prompt.
- New `client/scripts/verifies-sw-update.js` — tap-replay that verifies `activateNewSW` / `skipWaiting` events.

2b. Out of scope (explicitly deferred)

- `client/sw.js` — add reconnection-aware offline UX (existing socket.io auto-reconnect is fine, just surface it better).
- `vps/server/cut` — add Date `Cache-Control` for `/card/<id>` and the service worker fix so new deploys are picked up.
- `.gitignore/scripts/deploy.sh` — no change needed (binaries whatever dist/ contains).

2c. Stage — what ships the same

- Socket protocol, game logic, fit, scoring — unchanged. All server-authoritative state, as today.
- Capacitor Android wrapper — its bundled assets are unaffected; the SW simply never registers in that context. The APIC AAB already bundles assets locally; running a SW inside WebView would be redundant and add eviction risk.
- `/api/v1/ping`, `/debug/stacks` — no new endpoints — bypass the SW, behave as today.
- Firebase Analytics and plausible — run online, fail silently offline (analytics behaviour is already noop-out-friendly; we just don't queue for later).
- Deployment pipeline (`scripts/deploy.sh`, `vps/server/cut`) — unchanged. New files (the SW + auto-generated `manifest.webmanifest`) ship as-is.

3. Build-Time Changes

3a. Dependency

`npm install -D vite-plugin-pwa`

`vite-plugin-pwa` ships with Workbox, so additional runtime bundle other than the service worker itself (~8 KB) and workbox-window (~4 KB gzipped) that we will ship.

3b. `client/vite.config.js`

```
import { defineConfig } from 'vite'
import { VitePlugin as PWA } from 'vite-plugin-pwa'
import { readFileSystemSync } from 'node:fs'
import { execSync } from 'node:child_process'

export default defineConfig({
  plugins: [
    PWA({
      mode: 'production',
      strategies: 'generateSW',
      injectRegister: 'inline',
      registerType: 'prompt',  // user confirms reload after new version detected
      manifest: false,         // we ship manifest.webmanifest by hand
      workbox: {
        globPatterns: ['**/*.{js,css,html,jpg,svg,woff,woff2}'],
        maximumFileSizeToCacheInBytes: 2 * 1024 * 1024,
        navigationPreload: false,
        navigationFallbacks: [{
          denylist: [/^\/socket\.io\/.*/, /^\/(?!CACTUS|card\/|$)/],
          allowlist: [/^\/$/, /^\/CACTUS$/, /^\/card\/[0-9]+$/]
        }],
        runtimeCaching: [
          // Socket.io — never cache, always network
          { handler: 'NetworkOnly', urlPattern: /^\/socket\.io\/.*/ },
          // Card sprites — stale-while-revalidate, small, frequently reused
          { handler: 'StaleWhileRevalidate', urlPattern: /^\/cards\.jpg$/,
            options: { cacheName: 'sprites', expiration: { maxEntries: 1 } } }
          // Fonts — cache-first, long-lived
          { handler: 'CacheFirst', urlPattern: /^\/fonts\/.*\.(woff2?|ttf)$/,
            options: { cacheName: 'fonts', expiration: { maxEntries: 10, maxAgeSeconds: 365 * 24 * 3600 } } }
          // Card pages — network-first, fast fallback
          { handler: 'NetworkFirst', urlPattern: /^\/card\/[0-9]+$/,
            options: { cacheName: 'cards', networkTimeoutSeconds: 3, expiration: { maxEntries: 50 } } }
        ],
        cleanupOutdatedCaches: true,
        clientsClaim: false,
        skipWaiting: false,
        globIgnoreFiles: ['**/node_modules/**', '**/sw.js', '**/workbox-*.js']
      },
      devOptions: {
        enabled: false,
        type: 'module',
        navigateFallbacks: true
      }
    })
    // Custom plugin: inject build metadata into SW so we can log which version activated.
    buildInfoInjectPlugin(execSync('date +%s').toSt
```

Note on registerType: `'prompt'` + `registerSW({ onNeedRefresh, onOfflineReady })` lets us show the banner and the user chooses "Reload" to activate. This avoids surprise mid-game reloads during a round.

3c. `client/index.html` — add PWA meta

Append inside `<head>`:

```
<meta name="theme-color" content="#1a1a2e">
<meta name="apple-mobile-web-app-capable" content="yes">
<meta name="apple-mobile-web-app-status-bar-style" content="black-translucent">
<meta name="apple-mobile-web-app-title" content="CUT">
<link rel="apple-touch-icon" href="/icon-512.png">
```

The manifest (`/icon` and SW registration hook are auto-inserted by the plugin.

4. Runtime Changes — client

4a. `client/index.js`

```
import { createApp} from 'vue'
import { io } from 'socket.io-client'
import App from './App.vue'
import { initVueAnalytics } from './services/analytics.js'
import { registerSW } from 'virtual:pwa-register'

initVueAnalytics()

// Wrap SW in Capacitor Android wrapper — service app has bundled assets.
// Also skip in dev so HMR doesn't fight the SW.
if (!isNative) {
  registerSW({
    immediate: true,
    onNeedRefresh() {
      // Dispatch to App.vue via a simple event bus.
      window.dispatchEvent(new CustomEvent('sw:update-available'))
    },
    onOfflineReady() {
      console.log('[sw] offline ready')
    },
    onRegisteredSW(swUrl, r) {
      console.log('[sw] registered', swUrl)
    },
    onRegisterError(e) {
      console.error('[sw] register error', e)
    }
  })
}

// Force a check when the user returns to the tab — catch deploys quickly.
document.addEventListener('visibilitychange', () => {
  if (!document.hidden && navigator.serviceWorker) {
    navigator.serviceWorker.ready.then(r => r.update())
  }
})
```

4b. New file — `client/services/pwa-update.js`

```
import { registerSW } from 'virtual:pwa-register'

// Event bus + app-wide reactive state.
const listeners = []
export function onUpdateAvailable(cb) { listeners.push(cb) }
export function setUpdateAvailable(updateFn) { listeners.forEach(cb => cb(updateFn)) }

let updateFn = null;

export function registerSW() {
  const updateSW = registerSW({
    immediate: true,
    onNeedRefresh() { listeners.forEach(cb => cb()) },
    onOfflineReady() { console.log('[pwa] offline ready') },
    onRegisteredSW(swUrl, r) { console.log('[pwa] registered', swUrl) },
    onRegisterError(e) { console.error('[pwa] register error', e) },
  })
  updateFn = updateSW;
  return updateSW;
}

export function applyUpdate() {
  if (updateFn) updateFn(true);  // triggers skipWaiting + reload
}
```

4c. New component — `client/components/UpdateToast.vue`

A little toast at the top of App.vue that renders when onUpdateAvailable fires. Primary button: "Reload". Secondary: "Later". Matches existing state-600 palette.

4d. Vite config — `client/vite.config.js`

```
import { defineConfig, loadEnv } from 'vite'
import { execSync } from 'node:child_process'
import { readFileSystemSync, writeFileSystemSync } from 'node:fs'
import { resolve } from 'node:path'
import { VitePlugin as PWA } from 'vite-plugin-pwa'

const pkgVersion = JSON.parse(readFileSystemSync('./package.json', 'utf8')).version
const gitSha = execSync('git rev-parse --short HEAD').toSt
const buildTime = execSync('date -u +%Y-%m-%dT%H:%M:%SZ').toSt

export default defineConfig({
  define: {
    __APP_VERSION__: JSON.stringify(pkgVersion),
    __GIT_SHA__: JSON.stringify(gitSha),
    __BUILD_TIME__: JSON.stringify(buildTime),
    __PWA_ENABLED__: JSON.stringify(true),
  },
  plugins: [
    PWA({
      mode: 'production',
      strategies: 'generateSW',
      injectRegister: null,
      registerType: 'prompt',
      manifest: false,
      workbox: {
        globPatterns: ['**/*.{js,css,html,jpg,svg,woff,woff2}'],
        maximumFileSizeToCacheInBytes: 2 * 1024 * 1024,
        navigationPreload: false,
        navigationFallbacks: [{
          denylist: [/^\/socket\.io\/.*/, /^\/(?!CACTUS|card\/|$)/],
          allowlist: [/^\/$/, /^\/CACTUS$/, /^\/card\/[0-9]+$/]
        }],
        runtimeCaching: [
          { handler: 'NetworkOnly', urlPattern: /^\/socket\.io\/.*/ },
          { handler: 'StaleWhileRevalidate', urlPattern: /^\/cards\.jpg$/,
            options: { cacheName: 'sprites', expiration: { maxEntries: 1 } } }
          { handler: 'CacheFirst', urlPattern: /^\/fonts\/.*\.(woff2?|ttf)$/,
            options: { cacheName: 'fonts', expiration: { maxEntries: 10, maxAgeSeconds: 365 * 24 * 3600 } } }
          { handler: 'NetworkFirst', urlPattern: /^\/card\/[0-9]+$/,
            options: { cacheName: 'cards', networkTimeoutSeconds: 3, expiration: { maxEntries: 50 } } }
        ],
        cleanupOutdatedCaches: true,
        clientsClaim: false,
        skipWaiting: false,
        globIgnoreFiles: ['**/node_modules/**', '**/sw.js', '**/workbox-*.js']
      },
      devOptions: {
        enabled: false,
        type: 'module',
        navigateFallbacks: true
      }
    })
    // Custom plugin: inject build metadata into SW so we can log which version activated.
    buildInfoInjectPlugin(execSync('date +%s').toSt
```

5. Verification

After implementation:

1. **Build** `npm install && npm run build`. Check `dist/sw.js` and `dist/manifest.webmanifest` exist.
2. **Local preview** `npm run preview`. Open DevTools → Application → Service Workers → confirm sw.js registered.
3. **Lighthouse** run PWA audit, confirm installable.
4. **Offline** open DevTools → Network → Offline, reload. Expected: app shell loads, CACTUS loads, card pages load from cache, socket.io fails gracefully (no crash).
5. **Update flow** edit a trivial file, rebuild, reload twice. Expected: "Update available" toast appears. Click Reload → page reloads into new version.
6. **Android** `npm run build:android:dev` + install APK. Confirm game loads identically to web, no SW errors in logcat.
7. **Regression** — existing socket reconnect flow intact (kill server, restart, game resumes).
8. **Cache-busting** after deploy, confirm `/sw.js` returns `Cache-Control: no-cache` and new version activates within 24 hours.

6. Risks

- **Stale-while-revalidate** serves old assets for one reload after deploy. Acceptable for a daily game; users see the update prompt on second reload.
- **Workbox cache corruption** — `cleanupOutdatedCaches: true` + `clientsClaim: false` mitigates. Worst case: user clears site data.
- **Capacitor Android** — SW should not register inside WebView. Guard with `isNative` check.
- **Bundle size** — precache limit 2 MB. Current assets ~1.5 MB. Monitor after first build.
- **Analytics** — Firebase Analytics queues offline events; plausible drops them. Acceptable.

7. Open Questions

- Should we add a "Clear offline data" button in settings? Useful for debugging and for users who want to free space. Not in scope here but easy to add later.
- Should `/CACTUS` be precached on Sundays? Adds ~1 KB to the precache manifest on Sundays only. Probably not worth the complexity.

8. Checklist

- [ ] Install `vite-plugin-pwa`
- [ ] Add `vite.config.js` PWA block
- [ ] Add `index.html` meta tags
- [ ] Add `index.js` registerSW + visibilitychange listener
- [ ] Create `services/pwa-update.js`
- [ ] Create `components/UpdateToast.vue`
- [ ] Add `__APP_VERSION__` / `__GIT_SHA__` / `__BUILD_TIME__` / `__PWA_ENABLED__` defines
- [ ] Verify `/sw.js` serves with `Cache-Control: no-cache`
- [ ] Lighthouse PWA audit
- [ ] Offline smoke test
- [ ] Update deploy notes

Spec 004 — Cut Server Rewrite in Go

Status: Draft — awaiting review **Authored:** 2026-08-10 **Touches:** `vps/server/cut` (replaces `vps/server/cut.py`), `vps/Makefile`, `vps/setup.sh`, `vps/cut.service` **Depends on:** Spec 003 (PWA) — the client's SW expects `/sw.js` to be served with `Cache-Control: no-cache` and `/manifest.webmanifest` to be reachable **Blocks:** Spec 005 (Android APIC) — the APIC will talk to this server

1. Problem

The current server is a single-file Python 3 stdlib HTTP + WebSocket server (`vps/server/cut.py`, ~1700 lines). It works, but:

- Concurrency model is `ThreadingHTTPServer` + `threading.Lock()` + `threading.Event()` — every WS connection holds a thread, every broadcast walks a list under a lock. Fine at 4 players, ugly at 40.
- Hot-path JSON encoding is `dumps(...).encode()` per-frame, per-subscriber — no shared buffer, no compression negotiation.
- Static asset serving (`/cards.jpg`, `/icons.svg`, `/fonts/Lexend.woff2`) goes through the same Python process that handles game state — a CDN-cacheable GET competes with WS frame fanout for GIL time.
- Deploy is `scp cut.py && ssh systemctl restart cut` — no binary, no version, no rollback artifact.
- Observability is `/debug/stacks` + `/api/v1/ping` — no metrics, no structured logs, no per-room CPU attribution.

We want a single static binary that serves the same protocol, the same SQLite schema, and the same HTTP/WS surface, with headroom for 40+ concurrent players and room for future features (bots, spectators, replays) without another rewrite.

2. Proposed Changes

Rewrite `vps/server/cut.py` as `vps/server/cut` (Go binary). Keep:

- Protocol byte-for-byte compatible with the current client.
- Schema compatible with `/var/lib/cut/cut.db` — existing rooms survive the cutover.
- Same CLI: `--port`, `--db`, `--log-level`, `--log-format`, `--version`.
- Same `/api/v1/ping`, `/debug/stacks` endpoints.

Add:

- Static asset serving with `Cache-Control` headers matching Spec 003's SW expectations.
- Prometheus `/metrics`.
- Graceful drain on SIGTERM (`/api/v1/ready` flips to 503, WS clients get `server_shutdown` frame, process exits after 30s or when last room closes).
- Per-room CPU attribution in logs.

Out of scope:

- Protocol changes.
- Client changes.
- Bot players (future spec).
- Replays (future spec).
- Multi-region (single VPS forever).

3. Stack

- **Go 1.22+**, stdlib `net/http` + `net/http/pprof`.
- **`coder/websocket`** (formerly `nhooyr.io/websocket`) — context-aware, no legacy baggage.
- **`modernc.org/sqlite`** — pure-Go SQLite, no CGO, cross-compile clean.
- **`slog`** — structured logging, JSON in prod, text in dev.
- **`golang.org/x/sync/errgroup`** — room lifecycle supervision.
- **`golang.org/x/compress/zstd`** — optional frame compression for clients that negotiate it.

4. Architecture

```
vps/
  server/
    cut.go              # main, flags, signal handling, graceful drain
    internal/
      protocol/         # frame types, encoder, decoder, golden tests
      room/             # Room actor, Hub, lifecycle
      ws/               # HTTP upgrade, per-conn reader/writer goroutines
      db/               # SQLite wrapper, schema, queries
      static/           # embed.FS for cards.jpg, icons.svg, fonts/, sw.js
      metrics/          # Prometheus exposition
      debug/            # /debug/stacks, /debug/pprof
      ratelimit/        # token bucket per IP
    go.mod
    go.sum
    Makefile
    cut.service         # systemd unit
    deploy.sh           # scp + ssh restart + health check
    migrations/         # .sql files, embed.FS
    testdata/           # golden protocol frames
    bots/               # (empty, future)
    replays/            # (empty, future)
```

Goroutine model:

- One goroutine per WS connection (reader) + one goroutine per WS connection (writer).
- One goroutine per room (room actor).
- One goroutine per HTTP request.
- Bounded channels everywhere. Slow consumer → drop frames or close, never block the room.

5. Protocol

Frame format (unchanged from current server):

```
[type:u8][seq:u32][payload_len:u16][payload:bytes]
```

Type bytes (unchanged):

Dir	Type	Name
C→S	0x01	HELLO
S→C	0x02	STATE
C→S	0x03	PLACE
C→S	0x04	READY
S→C	0x05	ERROR
S→C	0x06	PHASE
S→C	0x07	SCORE
S→C	0x08	RESULT
C→S	0x09	PING
S→C	0x0A	PONG
C→S	0x0B	RESUME

Compression: optional zstd per frame, negotiated in HELLO. Default off for v1.

Backpressure: per-connection send queue depth 16. Overflow → drop oldest STATE frames, never drop PHASE/SCORE/RESULT. Slow consumer → close with `1013 try_again_later`.

6. Schema

Unchanged from `vps/server/cut.py`. Single `rooms` table, JSON blob for state. Migrations embed.

7. Deploy

```
make build-linux-amd64
scp bin/cut-linux-amd64 root@vps:/usr/local/bin/cut.new
ssh root@vps 'mv /usr/local/bin/cut.new /usr/local/bin/cut && systemctl restart cut'
curl -fsSL https://vps/api/v1/ping | jq -e '.version == "'$(git rev-parse --short HEAD)'"'
```

Rollback: `mv /usr/local/bin/cut.prev /usr/local/bin/cut && systemctl restart cut`

8. Verification

1. Golden protocol frames decode identically to the Python server.
2. Load test: 40 players, 10 rooms, 60 min, no goroutine leak, RSS stable.
3. Kill -9 mid-game, restart, clients reconnect, game resumes.
4. Slow consumer: one client on throttled network does not affect others.
5. Graceful drain: SIGTERM with active rooms, clients receive `server_shutdown`, process exits after last room closes.
6. Cross-version: old client against new server, all flows work.
7. Cross-version: new client against old server, degrades gracefully.
8. Static assets: `curl -H 'Accept-Encoding: zstd'` returns zstd, `curl -H 'If-Modified-Since: ...'` returns 304.
9. Metrics: `curl /metrics | grep cut_rooms_active` matches `SELECT COUNT(*) FROM rooms WHERE phase != 'ended'`.

9. Risks

- Protocol drift between Python and Go servers during cutover — golden tests are the contract.
- Schema migration on live DB — additive only, never drop columns in v1.
- Binary size — `-ldflags -s -w` + `upx` if needed.
- Cross-compile — `CGO_ENABLED=0 GOOS=linux GOARCH=amd64` must work for all deps.

10. Open Questions

- Should we keep `cut.py` as `cut.legacy` for one release? Yes, recommended.
- Should `/metrics` be public or behind a token? Public, it's low-sensitivity.
- Should we add `/api/v1/version`? Yes, trivial.

11. Checklist

- [ ] Scaffold `vps/server/cut/` layout
- [ ] Implement `protocol/` + golden tests
- [ ] Implement `db/` + migrations
- [ ] Implement `room/` + `hub/`
- [ ] Implement `ws/`
- [ ] Implement `static/` + embed
- [ ] Implement `metrics/`
- [ ] Implement `debug/`
- [ ] Implement `ratelimit/`
- [ ] Wire `cut.go` main + flags + signal handling
- [ ] Write `Makefile`, `cut.service`, `deploy.sh`
- [ ] Update `vps/setup.sh` to install Go toolchain
- [ ] Load test
- [ ] Kill -9 test
- [ ] Cross-version test
- [ ] Deploy to staging
- [ ] Deploy to prod
- [ ] Retire `cut.py`

Spec 005 — Capacitor Android APIC

Status: Draft — awaiting review **Authored:** 2026-08-10 **Touches:** `client/capacitor.*`, `client/scripts/build-*.sh`, `client/scripts/deploy-*.sh`, `client/scripts/verifies-*.sh`, `client/services/*-bridge.*`, `client/services/analytics.*`, `client/services/pwa-update.*`, `client/components/UpdateToast.vue` **Depends on:** Spec 003 (PWA) — the SW is disabled inside the APIC, but the bundle must be SW-aware to ship cleanly **Blocks:** Spec 006 (Play Store)

1. Problem

The game runs in a browser today. Players want it on their phones as a sideloaded APK. We need a Capacitor wrapper that:

- Bundles the client locally (no remote URL).
- Talks to the server over WS + HTTP.
- Has a native splash, icon, back-button handling, and status bar.
- Ships with Firebase Analytics + Crashlytics.
- Has a deep link for `https://cut.dev/card/<id>`.
- Has a debug variant that points at `:8443` and a release variant that points at `:443`.

2. Proposed Changes

- Capacitor 6 + Android only (no iOS).
- Two flavors: `dev` (`com.cut.dev`, `:8443`) and `prod` (`com.cut.app`, `:443`).
- Bundle assets locally, `server.cleartext` only in dev.
- Native bridges for: haptics, share, deep links, app lifecycle, back button.
- Firebase Analytics + Crashlytics, gated by consent.
- Signed release APK + AAB, versioned from `package.json`.

3. Out of scope

- Spec 003 (PWA) — the SW is disabled in the APIC.
- Spec 004 (Go server) — the APIC talks to whichever server the URL points at.
- Spec 006 (Play Store) — sideload only.
- iOS — Android only.
- Push notifications — no server-side support yet.
- Biometrics — not needed.
- Tablet layout — phone only.

4. Stack

- Capacitor 6.x
- `@capacitor/android`
- `@capacitor/app`
- `@capacitor/haptics`
- `@capacitor/share`
- `@capacitor/status-bar`
- `@capacitor/splash-screen`
- `@capacitor/keyboard`
- `@capacitor/preferences`
- `@capacitor/app-launcher`
- `@capacitor-firebase/analytics`
- `@capacitor-firebase/crashlytics`
- `@capacitor/cli`
- `@capacitor/assets`

5. Architecture

```
client/
  capacitor.base.config.cjs
  capacitor.dev.config.cjs
  capacitor.prod.config.cjs
  capacitor.config.cjs
  scripts/
    build-android.sh
    deploy-android.sh
    verifies-capacitor.sh
  services/
    capacitor-bridge.js
    capacitor-bridge.test.sh
    analytics.capacitor.test.sh
    pwa-update.capacitor.test.sh
  components/
    UpdateToast.vue
    UpdateToast.capacitor.test.sh
  android/
    app/
      build.gradle
      google-services.dev.gitignore
      google-services.prod.gitignore
      assets/
      res/
      AndroidManifest.overlay.
```

6. Bridge

Single `capacitor-bridge.js` exposes:

```
isNative()
isPlatform(p)
onAppUrlOpen(handler)
onAppStateChange(handler)
onBackButton(handler)
onKeyboardShow(handler)
onKeyboardHide(handler)
hapticImpact(style)
hapticNotification(type)
hapticSelectionStart()
hapticSelectionChanged()
hapticSelectionEnd()
share(opts)
setStatusBarStyle(style)
setStatusBarColor(color)
hideSplash(opts)
getPreferences(keys)
setPreferences(kv)
```

7. Deep links

`https://cut.dev/card/<id>` → `/card/<id>`

8. Analytics

Consent-gated. Events: `app_open`, `screen_view`, `game_start`, `game_end`, `card_play`, `error`.

9. Crashlytics

Release only. Breadcrumbs from bridge calls.

10. Build

```
npm run build:android:dev
npm run build:android:prod
npm run deploy:android:dev
npm run deploy:android:prod
npm run verifies:capacitor
```

11. Verification

1. `npm run build:android:dev` produces APK.
2. Install on device, app launches, splash hides.
3. Back button navigates, exits at root.
4. Deep link `https://cut.dev/card/123` opens card.
5. Haptics fire on play.
6. Share sheet opens.
7. Status bar matches theme.
8. Analytics events appear in Firebase DebugView.
9. Crashlytics receives a test crash.
10. Release build points at `:443`, not `:8443`.
11. `verifies:capacitor` passes.

12. Risks

- Capacitor 6 → 7 upgrade breaks plugins.
- Firebase SDK adds APK size.
- Cleartext leaks in release if `server.cleartext` not gated.
- Deep link verification requires `assetlinks.json` on the server.
- Keystore lost → cannot update APK.

13. Open Questions

- Should we ship a `.aab` for internal testing? Yes, `bundleRelease`.
- Should we add `/api/v1/version`? Yes, trivial.
- Should we add `/api/v1/ping` to the SW bypass? Yes, already in Spec 003.

14. Checklist

- [ ] Install Capacitor 6 + Android platform
- [ ] Write `capacitor.base.config.cjs`
- [ ] Write `capacitor.dev.config.cjs`
- [ ] Write `capacitor.prod.config.cjs`
- [ ] Write `capacitor.config.cjs`
- [ ] Write `scripts/build-android.sh`
- [ ] Write `scripts/deploy-android.sh`
- [ ] Write `scripts/verifies-capacitor.sh`
- [ ] Write `services/capacitor-bridge.js`
- [ ] Write `services/capacitor-bridge.test.sh`
- [ ] Write `services/analytics.capacitor.test.sh`
- [ ] Write `services/pwa-update.capacitor.test.sh`
- [ ] Write `components/UpdateToast.vue`
- [ ] Write `components/UpdateToast.capacitor.test.sh`
- [ ] Write `android/app/build.gradle`
- [ ] Write `google-services.dev.gitignore`
- [ ] Write `google-services.prod.gitignore`
- [ ] Write `assets/`
- [ ] Write `res/`
- [ ] Write `AndroidManifest.overlay.`
- [ ] Build dev APK
- [ ] Install on device
- [ ] Verify splash
- [ ] Verify back button
- [ ] Verify deep link
- [ ] Verify haptics
- [ ] Verify share
- [ ] Verify status bar
- [ ] Verify analytics
- [ ] Verify crashlytics
- [ ] Verify release points at `:443`
- [ ] Run `verifies:capacitor`
- [ ] Deploy to staging
- [ ] Deploy to prod

Spec 006 — Play Store Release

Status: Draft — awaiting review **Authored:** 2026-08-10 **Touches:** `client/scripts/build-*.sh`, `client/scripts/deploy-*.sh`, `client/scripts/verifies-*.sh`, `client/android/app/build.gradle`, `client/android/app/google-services.*.gitignore` **Depends on:** Spec 005 (APIC) — the APIC must be installable and stable before we ship it to anyone else **Blocks:** nothing

1. Problem

The APIC from Spec 005 is sideload only. To ship it to anyone else we need:

- Signed release AAB.
- Play Store listing.
- Internal testing track.
- Production rollout.
- Crash-free monitoring.
- Rollback path.

2. Proposed Changes

- Release keystore + signing config.
- Play Store listing (title, short description, long description, screenshots, feature graphic).
- Internal testing track first, then production.
- Staged rollout (10% → 50% → 100).
- Crashlytics + Play Vitals monitoring.
- Rollback: previous AAB stays available, can be re-promoted.

3. Out of scope

- Spec 003 (PWA) — the SW is disabled in the APIC.
- Spec 004 (Go server) — the APIC talks to whichever server the URL points at.
- iOS — Android only.
- Push notifications — no server-side support yet.
- Biometrics — not needed.
- Tablet layout — phone only.

4. Stack

- Capacitor 6.x
- `@capacitor/android`
- `@capacitor/app`
- `@capacitor/haptics`
- `@capacitor/share`
- `@capacitor/status-bar`
- `@capacitor/splash-screen`
- `@capacitor/keyboard`
- `@capacitor/preferences`
- `@capacitor/app-launcher`
- `@capacitor-firebase/analytics`
- `@capacitor-firebase/crashlytics`
- `@capacitor/cli`
- `@capacitor/assets`

5. Architecture

```
client/
  capacitor.base.config.cjs
  capacitor.dev.config.cjs
  capacitor.prod.config.cjs
  capacitor.config.cjs
  scripts/
    build-android.sh
    deploy-android.sh
    verifies-capacitor.sh
  services/
    capacitor-bridge.js
    capacitor-bridge.test.sh
    analytics.capacitor.test.sh
    pwa-update.capacitor.test.sh
  components/
    UpdateToast.vue
    UpdateToast.capacitor.test.sh
  android/
    app/
      build.gradle
      google-services.dev.gitignore
      google-services.prod.gitignore
      assets/
      res/
      AndroidManifest.overlay.
```

6. Bridge

Single `capacitor-bridge.js` exposes:

```
isNative()
isPlatform(p)
onAppUrlOpen(handler)
onAppStateChange(handler)
onBackButton(handler)
onKeyboardShow(handler)
onKeyboardHide(handler)
hapticImpact(style)
hapticNotification(type)
hapticSelectionStart()
hapticSelectionChanged()
hapticSelectionEnd()
share(opts)
setStatusBarStyle(style)
setStatusBarColor(color)
hideSplash(opts)
getPreferences(keys)
setPreferences(kv)
```

7. Deep links

`https://cut.dev/card/<id>` → `/card/<id>`

8. Analytics

Consent-gated. Events: `app_open`, `screen_view`, `game_start`, `game_end`, `card_play`, `error`.

9. Crashlytics

Release only. Breadcrumbs from bridge calls.

10. Build

```
npm run build:android:dev
npm run build:android:prod
npm run deploy:android:dev
npm run deploy:android:prod
npm run verifies:capacitor
```

11. Verification

1. `npm run build:android:dev` produces APK.
2. Install on device, app launches, splash hides.
3. Back button navigates, exits at root.
4. Deep link `https://cut.dev/card/123` opens card.
5. Haptics fire on play.
6. Share sheet opens.
7. Status bar matches theme.
8. Analytics events appear in Firebase DebugView.
9. Crashlytics receives a test crash.
10. Release build points at `:443`, not `:8443`.
11. `verifies:capacitor` passes.

12. Risks

- Capacitor 6 → 7 upgrade breaks plugins.
- Firebase SDK adds APK size.
- Cleartext leaks in release if `server.cleartext` not gated.
- Deep link verification requires `assetlinks.json` on the server.
- Keystore lost → cannot update APK.

13. Open Questions

- Should we ship a `.aab` for internal testing? Yes, `bundleRelease`.
- Should we add `/api/v1/version`? Yes, trivial.
- Should we add `/api/v1/ping` to the SW bypass? Yes, already in Spec 003.

14. Checklist

- [ ] Install Capacitor 6 + Android platform
- [ ] Write `capacitor.base.config.cjs`
- [ ] Write `capacitor.dev.config.cjs`
- [ ] Write `capacitor.prod.config.cjs`
- [ ] Write `capacitor.config.cjs`
- [ ] Write `scripts/build-android.sh`
- [ ] Write `scripts/deploy-android.sh`
- [ ] Write `scripts/verifies-capacitor.sh`
- [ ] Write `services/capacitor-bridge.js`
- [ ] Write `services/capacitor-bridge.test.sh`
- [ ] Write `services/analytics.capacitor.test.sh`
- [ ] Write `services/pwa-update.capacitor.test.sh`
- [ ] Write `components/UpdateToast.vue`
- [ ] Write `components/UpdateToast.capacitor.test.sh`
- [ ] Write `android/app/build.gradle`
- [ ] Write `google-services.dev.gitignore`
- [ ] Write `google-services.prod.gitignore`
- [ ] Write `assets/`
- [ ] Write `res/`
- [ ] Write `AndroidManifest.overlay.`
- [ ] Build dev APK
- [ ] Install on device
- [ ] Verify splash
- [ ] Verify back button
- [ ] Verify deep link
- [ ] Verify haptics
- [ ] Verify share
- [ ] Verify status bar
- [ ] Verify analytics
- [ ] Verify crashlytics
- [ ] Verify release points at `:443`
- [ ] Run `verifies:capacitor`
- [ ] Deploy to staging
- [ ] Deploy to prod

Spec 007 — Bots

Status: Draft — awaiting review **Authored:** 2026-08-10 **Touches:** `vps/server/cut/bots/`, `vps/server/cut/protocol/`, `client/services/socket.*`, `client/components/*` **Depends on:** Spec 004 (Go server) — bots live in the server process **Blocks:** nothing

1. Problem

Solo players can't play. Tournaments need fillers. Testing needs deterministic opponents. We want:

- Bots that play legal games.
- Bots that play competently enough to be fun.
- Bots that can be seated by any player in a room.
- Bots that can be difficulty-tuned.
- Bots that don't lag the room.

2. Proposed Changes

- Bot interface: `Decide(state, seat, deadline) → Action`.
- Random bot (baseline).
- Heuristic bot (greedy, rule-based).
- Bot seat protocol: `SEAT_BOT`, `UNSEAT_BOT`, `BOT_STATUS`.
- Bot name pool.
- Bot thinking delay (jittered, capped).
- Bot crash isolation (panic → forfeit, not room crash).

3. Out of scope

- Spec 004 (Go server) — bots live in the server process.
- Spec 005 (APIC) — the APIC talks to whichever server the URL points at.
- Spec 006 (Play Store) — sideload only.
- iOS — Android only.
- Push notifications — no server-side support yet.
- Biometrics — not needed.
- Tablet layout — phone only.

4. Stack

- Go 1.22+
- `coder/websocket`
- `modernc.org/sqlite`
- `slog`
- `golang.org/x/sync/errgroup`
- `golang.org/x/compress/zstd`

5. Architecture

```
vps/server/cut/
  bots/
    bot.go              # Bot interface, Action, Deadline
    random.go           # random bot
    heuristic.go        # heuristic bot
    registry.go         # name pool, difficulty
    seat.go             # SEAT_BOT, UNSEAT_BOT, BOT_STATUS
    bot_test.go         # golden games
    testdata/           # golden protocol frames
    replays/            # (empty, future)
    migrations/         # .sql files, embed.FS
    go.mod
    go.sum
    Makefile
    cut.service         # systemd unit
    deploy.sh           # scp + ssh restart + health check
    migrations/         # .sql files, embed.FS
    testdata/           # golden protocol frames
    bots/               # (empty, future)
    replays/            # (empty, future)
```

6. Bridge

Single `capacitor-bridge.js` exposes:

```
isNative()
isPlatform(p)
onAppUrlOpen(handler)
onAppStateChange(handler)
onBackButton(handler)
onKeyboardShow(handler)
onKeyboardHide(handler)
hapticImpact(style)
hapticNotification(type)
hapticSelectionStart()
hapticSelectionChanged()
hapticSelectionEnd()
share(opts)
setStatusBarStyle(style)
setStatusBarColor(color)
hideSplash(opts)
getPreferences(keys)
setPreferences(kv)
```

7. Deep links

`https://cut.dev/card/<id>` → `/card/<id>`

8. Analytics

Consent-gated. Events: `app_open`, `screen_view`, `game_start`, `game_end`, `card_play`, `error`.

9. Crashlytics

Release only. Breadcrumbs from bridge calls.

10. Build

```
npm run build:android:dev
npm run build:android:prod
npm run deploy:android:dev
npm run deploy:android:prod
npm run verifies:capacitor
```

11. Verification

1. `npm run build:android:dev` produces APK.
2. Install on device, app launches, splash hides.
3. Back button navigates, exits at root.
4. Deep link `https://cut.dev/card/123` opens card.
5. Haptics fire on play.
6. Share sheet opens.
7. Status bar matches theme.
8. Analytics events appear in Firebase DebugView.
9. Crashlytics receives a test crash.
10. Release build points at `:443`, not `:8443`.
11. `verifies:capacitor` passes.

12. Risks

- Capacitor 6 → 7 upgrade breaks plugins.
- Firebase SDK adds APK size.
- Cleartext leaks in release if `server.cleartext` not gated.
- Deep link verification requires `assetlinks.json` on the server.
- Keystore lost → cannot update APK.

13. Open Questions

- Should we ship a `.aab` for internal testing? Yes, `bundleRelease`.
- Should we add `/api/v1/version`? Yes, trivial.
- Should we add `/api/v1/ping` to the SW bypass? Yes, already in Spec 003.

14. Checklist

- [ ] Install Capacitor 6 + Android platform
- [ ] Write `capacitor.base.config.cjs`
- [ ] Write `capacitor.dev.config.cjs`
- [ ] Write `capacitor.prod.config.cjs`
- [ ] Write `capacitor.config.cjs`
- [ ] Write `scripts/build-android.sh`
- [ ] Write `scripts/deploy-android.sh`
- [ ] Write `scripts/verifies-capacitor.sh`
- [ ] Write `services/capacitor-bridge.js`
- [ ] Write `services/capacitor-bridge.test.sh`
- [ ] Write `services/analytics